TP8 FoodStore - Base de la tablet e integración

Bases de Datos II - Unidad 8
Franco Sampieri, Mauro Di Pietro, Tomas Martinez y Renzo Victoria

1. Base local de la tablet

La tablet guarda una copia pequeña del catálogo para trabajar sin conexión. Incluye identificador, nombre, precio en centavos, stock visto, disponibilidad, baja lógica y fecha de captura. No copia contraseñas, mails, celulares, imágenes ni ventas. PostgreSQL sigue siendo la fuente de verdad del precio, el stock y los pedidos confirmados.

En **movil/schema_sqlite.sql** se crearon cuatro tablas STRICT: producto_cache, pedido_borrador, item_borrador y cola_sync. La tabla persistente snapshot_servidor recibe el CSV. Hay claves foráneas, restricciones CHECK y un índice parcial que admite un único envío pendiente por borrador.

La primera instantánea contenía 25 productos, incluidos tres dados de baja. El catálogo disponible mostró 21 productos; después de importar la segunda instantánea mostró 20. La consulta C1 se ejecutó localmente, sin consultar PostgreSQL.

El borrador, su ítem y su solicitud se guardan dentro de una transacción. En C7, una cantidad cero produjo un error CHECK y un producto inexistente produjo un error de clave foránea. En ambos casos el ROLLBACK dejó cero filas nuevas.

2. Instantánea y semáforo

La vista PostgreSQL **v_snapshot_movil** exporta el catálogo, incluidos los productos dados de baja. Se cargó el CSV en snapshot_servidor y el UPSERT actualizó producto_cache sin borrar los ítems que hacen referencia a productos eliminados.

En la copia de pruebas **foodstore_u8**, la Muzzarella pasó de $2.400 a $2.500 y la Fugazzeta recibió baja lógica. Los dos CSV conservaron 25 IDs y el mismo stock total: 785 unidades.

Caso	Resultado verificado
C2 - precio	CONFLICTO por 240000 frente a 250000 centavos. Tras aceptar el precio y crear otra solicitud, la nueva quedó SIMULADO_APROBADO; la original conservó el conflicto.
C3 - stock	RECHAZADO: se solicitaron 16 unidades y el stock de la instantánea era 15.
C4 - baja	RECHAZADO: la Fugazzeta quedó eliminada. Su ítem permaneció guardado y dejó de aparecer en el catálogo visible.
C5 - sin cambios	SIMULADO_APROBADO: coincidieron precio, disponibilidad y stock.
C6 - UUID	El segundo envío con el mismo UUID falló por unicidad. Quedó una sola fila y el estado previo no cambió.

SIMULADO_APROBADO no representa una venta: la tablet no llamó a sp_crear_pedido ni descontó stock. En PostgreSQL, la comprobación final de foodstore_u8 devolvió **21 pedidos y 785 unidades**, igual que antes de la simulación.

3. Integración y mantenimiento

Motor	Responsabilidad
PostgreSQL	Catálogo oficial, usuarios, pedidos y stock. Decide precios y ventas.
MongoDB	Copia documental de cinco pedidos de la Unidad 7; sirve para lectura, no para confirmar ventas.
SQLite	Catálogo copiado, borradores y solicitudes pendientes de la tablet; permite trabajar sin señal.

El respaldo principal sigue siendo el de PostgreSQL en formato custom, comprobado mediante una restauración de prueba. Se conservan también la exportación de MongoDB y los archivos SQLite que tengan borradores pendientes. Tras restaurar PostgreSQL desde un respaldo antiguo, cada tablet debe renovar su instantánea y revisar las solicitudes pendientes antes de reenviarlas. Los rechazos y conflictos se revisan después de cada sincronización y al cierre del día.

4. Uso y revisión de IA

Kiro redactó requisitos EARS, diseño y tareas. OpenCode generó los scripts y, en modo Plan, revisó el proyecto de las unidades 1 a 8. Codex se usó como herramienta adicional para contrastar los CSV, revisar el SQL y ejecutar pruebas locales de SQLite. El equipo aceptó las decisiones después de contrastarlas con el código y los resultados, y dejó el detalle en la bitácora y la declaración de IA del repositorio.

De la revisión global se desarrollaron tres hallazgos de distintas unidades:

Hallazgo	Decisión y evidencia
H2 (U1): precio directo	Se aceptó. Antes, un INSERT directo con 0.01 guardó 0.01 de precio y subtotal. OpenCode corrigió fn_set_subtotal; después, el mismo intento guardó 2500.00 y 5000.00. Al cambiar cantidad, subtotal = cantidad × precio_unitario resultó verdadero.
H4 (U3): índice parcial	Se descartó que fuera inútil en general. En TP5, la consulta con disponible = FALSE y eliminado = FALSE usó el índice parcial y pasó de 31,581 ms a 2,767 ms. Se corrigió el comentario SQL que omitía el segundo filtro.
H6 (U8): borrador vacío	Se aceptó y corrigió. Antes quedaba PENDIENTE sin motivo; después, RECHAZADO con motivo 'Borrador sin ítems'. Las pruebas C2-C6 conservaron sus estados.

Además, se corrigió H1 (U1): el trigger ahora impide mover un detalle a otro pedido o producto. La prueba de traslado produjo la excepción esperada. Otros hallazgos aceptados se registraron como límites de integridad o reutilización; se descartaron dos observaciones que no constituían defectos funcionales. Las pruebas de PostgreSQL se hicieron en transacciones revertidas para no alterar la copia de trabajo.

Repositorio: github.com/francosampieri/TP2_BDD2_FoodStore